1. Title Page

Optimising NP-Hard Constraints: A Dynamic Programming Approach to the Travelling Salesman Problem in Sustainable Logistics

Anjini Pandey

Vellore Institute of Technology (VIT)

Design and Analysis of Algorithms

Faculty Name: Padmavathy T

April 3, 2026

---

2. Author Details

Anjini Pandey

Department of Computer Science and Engineering

Email ID: anjini.pandey2024@vitstudent.ac.in

---

3. Abstract

The Travelling Salesman Problem (TSP) is a highly complex computational challenge that directly impacts how we plan delivery routes and logistics. Finding an exact route by checking every possible path requires too much time because the number of options grows exponentially. To solve this efficiently, we can use Dynamic Programming, specifically the Held-Karp algorithm. This method breaks the massive problem down into smaller, overlapping parts, finding the most efficient route without guessing. We tested this approach using a 4-node matrix and the G (P, S) recurrence relation, successfully calculating a perfectly optimised route. By applying Dynamic Programming to TSP, logistics companies can significantly reduce fuel consumption and carbon emissions. In conclusion, while TSP remains a difficult constraint, Dynamic Programming provides a rigorous, mathematical solution to make sustainable deliveries more efficient.

---

4. Keywords

Travelling Salesman Problem, Dynamic Programming, Held-Karp Algorithm, NP-Hard, Sustainable Logistics, Time Complexity, Memoisation

---

5. Introduction

In computer science, problems are grouped by how they scale to solve as the input size grows.

The class P has problems that can be solved quickly, while NP has problems where a solution can be checked quickly.

The Travelling Salesman Problem asks for the route that visits a set of cities just once and returns to the starting point.

The Travelling Salesman Problem is very hard to solve because it takes a very long time to compute the answer as the number of cities increases.

This creates problems for modern systems engineering.

We want to look at one way to solve the Travelling Salesman Problem, which is called Dynamic Programming.

By using computer memory, we can solve the problem much faster.

In this paper, we will show how we can use this method to find the solution and use it to make local delivery routes more efficient, like waste collection in cities.

The goal is to make these routes better and help the environment.

We will see how this method can be used in real-life situations.

The Travelling Salesman Problem and Dynamic Programming are key to solving these problems.

---

6. Literature Review

Historically, the computer approach to the Travelling Salesman Problem or TSP is about balancing how long it takes to solve with how accurate the solution is.

The Brute Force method always finds the solution, but it is slow, taking a very long time to solve even small problems.

To get around this, many systems use algorithms that do not always find the best solution, like the Minimum Spanning Tree or the Christofides algorithm.

These algorithms work quickly. Only guarantee a solution that is not much worse than the best one, which can lead to some waste.

In 1962, Michael Held and Richard Karp came up with a way to solve TSP using dynamic programming that works well for smaller problems.

The Held-Karp algorithm makes the problem easier to solve and remembers information, making it much faster.

Even though it is still quite complex, it guarantees finding the possible route, which makes it very useful for important routing problems where there are not too many places to visit.

The Held-Karp algorithm has a time complexity of $O(2^n)$, which is much better than the brute-force method.

This makes the Held-Karp algorithm a good choice for solving TSP.

---

7. Methodology

This paper uses the Dynamic Programming method to solve the Travelling Salesman Problem with the Held-Karp algorithm. The Travelling Salesman Problem is solved with this approach. It does this by breaking down the Travelling Salesman Problem into parts that overlap and remembering the answers to these smaller parts of the Travelling Salesman Problem. This way, the Travelling Salesman Problem is solved without guessing.

Mathematical Model:

Let the problem be defined by a set of checkpoints. Let w (P, j) represent the distance (weight) between checkpoint P and checkpoint j.

The problem is solved using the following recurrence relation:

$$\mathbf{G\ (P, S) = \min_{j \in S}\{w\ (P, j)\ +\ G\ (j, S\ -\ \{j\}\}}$$

Where:

- P is the current starting vertex (or point).
- S is the set of remaining unvisited vertices.
- G (P, S) represents the minimum cost to visit all vertices in set S starting from vertex P and finally returning to the original starting depot (Node A).

The base case for the recurrence, where the set of unvisited nodes is empty (S = ∅), is strictly the distance from the final node back to the origin:

G (P, ∅) = w (P, A)

---

8. Implementation / Analysis

To evaluate this mathematical model, the dynamic programming recurrence is applied to a 4-node sustainable logistics network. The vertices are {A, B, C, D}, corresponding to four distinct waste collection checkpoints. Node A serves as the starting matrix/depot.

Adjacency Matrix Representation

To accurately compute the DP paths, the network weights are represented by the following distance matrix:

	A	B	C	D
A	0	16	11	6
B	8	0	13	16
C	4	7	0	9
D	5	12	2	0

Held-Karp Algorithm Execution: Step-by-Step

Step 1: Base Cases (|S| = 0)

Calculate the cost of returning to the starting depot (Node A) from all other individual nodes:

- G (B, ∅) = w (B, A) = 8
- G (C, ∅) = w (C, A) = 4
- G (D, ∅) = w (D, A) = 5

Step 2: Subproblems of Size 1 (|S| = 1)

Calculate the minimum cost of visiting one remaining unvisited node and then returning to the depot:

- G (B, {C}) = w (B, C) + G (C, ∅) = 13 + 4= 17
- G (B, {D}) = w (B, D) + G (D, ∅) = 16 + 5 = 21
- G (C, {B}) = w (C, B) + G (B, ∅) = 7 + 8 = 15
- G (C, {D}) = w (C, D) + G (D, ∅) = 9 + 5 = 14
- G (D, {B}) = w (D, B) + G (B, ∅) = 12 + 8 = 20
- G (D, {C}) = w (D, C) + G (C, ∅) = 2 + 4 = 6

Step 3: Subproblems of Size 2 (|S| = 2)

Calculate the minimum cost of visiting two remaining unvisited nodes before returning to the depot, utilising the memorised values from Step 2:

- $\mathbf{G}\ (\mathbf{B}, \{\mathbf{C}, \mathbf{D}\}) = \boldsymbol{min} \begin{cases} \mathbf{w}(\mathbf{B}, \mathbf{C}) + \mathbf{G}(\mathbf{C}, \{\mathbf{D}\}) \\ \mathbf{w}(\mathbf{B}, \mathbf{D}) + \mathbf{G}(\mathbf{D}, \{\mathbf{C}\}) \end{cases}$
  - $\mathbf{G}\ (\mathbf{B}, \{\mathbf{C}, \mathbf{D}\}) = \boldsymbol{min} \begin{cases} \mathbf{13 + 14 = 27} \\ \mathbf{16 + 6 = 22} \end{cases}$
  - $\mathbf{G}\ (\mathbf{B}, \{\mathbf{C}, \mathbf{D}\}) = \mathbf{22}$
- $\mathbf{G}\ (\mathbf{C}, \{\mathbf{B}, \mathbf{D}\}) = \boldsymbol{min} \begin{cases} \mathbf{w}(\mathbf{C}, \mathbf{B}) + \mathbf{G}(\mathbf{B}, \{\mathbf{D}\}) \\ \mathbf{w}(\mathbf{C}, \mathbf{D}) + \mathbf{G}(\mathbf{D}, \{\mathbf{B}\}) \end{cases}$
  - $\mathbf{G}\ (\mathbf{C}, \{\mathbf{B}, \mathbf{D}\}) = \boldsymbol{min} \begin{cases} \mathbf{7 + 21 = 28} \\ \mathbf{9 + 20 = 29} \end{cases}$
  - $\mathbf{G}\ (\mathbf{C}, \{\mathbf{B}, \mathbf{D}\}) = \mathbf{28}$
- $\mathbf{G}\ (\mathbf{D}, \{\mathbf{B}, \mathbf{C}\}) = \boldsymbol{min} \begin{cases} \mathbf{w}(\mathbf{D}, \mathbf{B}) + \mathbf{G}(\mathbf{B}, \{\mathbf{C}\}) \\ \mathbf{w}(\mathbf{D}, \mathbf{C}) + \mathbf{G}(\mathbf{C}, \{\mathbf{B}\}) \end{cases}$
  - $\mathbf{G}\ (\mathbf{D}, \{\mathbf{B}, \mathbf{C}\}) = \boldsymbol{min} \begin{cases} \mathbf{12 + 17 = 29} \\ \mathbf{2 + 15 = 17} \end{cases}$
  - $\mathbf{G}\ (\mathbf{D}, \{\mathbf{B}, \mathbf{C}\}) = \mathbf{17}$

Step 4: Final Computation (|S| = 3)

Calculate the optimal overarching tour starting from Node A and visiting all nodes in the set {B, C, D} before returning to A:

$$\mathbf{G(A,\{B,C,D\})} = \boldsymbol{min}\begin{cases} \mathbf{w(A,B)\ +\ G(B,\{C,D\})} \\ \mathbf{w(A,C)\ +\ G(C,\{B,D\})} \\ \mathbf{w(A,D)\ +\ G(D,\{B,C\})} \end{cases}$$

Substituting the previously memorised values from Step 3:

$$\mathbf{G(A,\{B,C,D\})} = \boldsymbol{min}\begin{cases} \mathbf{16+22=38} \\ \mathbf{11+28=39} \\ \mathbf{6+17=23} \end{cases}$$

$$\mathbf{G(A,\{B,C,D\})} = \mathbf{23}$$

---

9. Results

The math works out to give us a tour with a total distance of 23. We used the Held-Karp algorithm to avoid doing the work again, like calculating the cost from Node C to Node D multiple times. The Held-Karp algorithm helps us with this. It took some time to get the answer. We were able to keep it from getting too crazy by using something called memoisation. This helps when we are working with a group of places, like a local area. The Held-Karp algorithm and memoisation really help with routing graphs.

---

10. Discussion

The results show that dynamic programming is very good when we need to be exact about how we use resources and plan logistics.

The good thing about this method is that it gives us the answer. When we are talking about logistics, finding the perfect route. Not just a close guess. Means we use the least amount of fuel and make the least amount of carbon emissions.

The bad thing about this method is that it needs a lot of space to store information. The Held-Karp algorithm must keep all the answers to the problems it solves, so it does not have to solve them again. This means it needs a lot of memory. The amount of memory it needs grows fast. This works well when we are dealing with an area. Like a few cities. But it uses too much memory when we are dealing with a lot of cities, so we cannot use it for big supply chains without breaking them down into smaller parts.

Dynamic programming is great for logistics. It has its downsides, like the space it needs to store information. Dynamic programming is very useful, for planning routes and using resources. We must be careful when using it for big problems.

---

11. Conclusion

This paper successfully demonstrates the application of Dynamic Programming to resolve the Travelling Salesman Problem within the context of sustainable logistics. By utilising the Held-Karp algorithm, the routing logic achieves an exact mathematical optimisation, providing a provably perfect operational route of 23 for the modelled network. While constrained by exponential memory requirements, DP remains the superior algorithmic choice for contained routing problems where environmental and operational efficiency must be maximised.

Future Scope: Future advancements could focus on hybrid architectures, partitioning massive logistical graphs into localised clusters. DP could perfectly optimise routing within each cluster, while polynomial approximation algorithms handle the macro-routing between clusters, blending exact precision with broad scalability.

---

12. References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to Algorithms* (3rd ed.). MIT Press.
2. Held, M., & Karp, R. M. (1962). "A dynamic programming approach to sequencing problems". *Journal of the Society for Industrial and Applied Mathematics*, 10(1), 196-210.
3. Bellman, R. (1962). "Dynamic programming treatment of the travelling salesman problem". *Journal of the ACM*, 9(1), 61-63.
4. Vazirani, V. V. (2001). *Approximation Algorithms*. Springer-Verlag.